# Gap Analysis — Built vs. Specified

Comparing the current state of the codebase against the six planning documents. Verdict up front: **the gap is large, not minimum** — the six documents describe a substantially bigger system than what currently exists, particularly on refunds/returns/delivery and on the entire frontend.

---

## 1. Backend — what's actually built and matches the spec

These are done, tested, and consistent with the API Specification:

- Auth: sign-up, sign-in, session, role-gated (`customer`/`admin`), rate limited
- `GET /api/products` — search, filter, pagination
- `GET /api/cart`, `POST /api/cart` (upsert)
- `POST /api/orders` (checkout, transactional, stock-safe), `GET /api/orders`
- `POST /api/orders/{id}/pay` — Paystack checkout initialization
- Admin: create product, update inventory, update order status, sales dashboard
- Rate limiting across auth + custom routes

**This is a solid, correctly-built subset — but it's roughly the scope of the system as it existed *before* the returns/refunds/delivery/guest-cart conversations happened.** Everything since then is net-new, not yet started.

## 2. Backend — specified but not built

| Area | What's missing | Size |
|---|---|---|
| **Payment webhook** | `/api/webhooks/paystack` doesn't exist. Nothing currently marks an order `paid` automatically. This is the single most urgent gap — without it, the whole automatic-payment path is incomplete even though checkout initialization works. | **Medium, urgent** |
| **Returns workflow** | `returns`, `return_items` tables don't exist yet. No request/approve/reject/receive/inspect endpoints. Entire flow from the Ops Spec is unbuilt. | **Large** |
| **Refunds** | `refunds` table doesn't exist. No refund-triggering logic, no Paystack refund API integration. | **Large** |
| **Deliveries** | `deliveries` table doesn't exist. No admin endpoint to update delivery sub-status. | **Medium** |
| **Manual payments** | No `mark-paid` endpoint, no `provider: "manual"` support on `payments`, no admin attribution columns. | **Small–Medium** |
| **Notifications** | `notifications` table doesn't exist. No list/mark-read endpoints. | **Medium** |
| **Guest cart** | `carts.userId` is currently `NOT NULL` — the guest-cart schema change (nullable `userId` + new `guestId` column) hasn't been applied. No merge-on-sign-in logic exists. | **Medium** |
| **Categories API** | No endpoints at all — currently managed via raw SQL. | **Small** |
| **Addresses API** | No endpoints at all — currently managed via raw SQL. | **Small** |
| **Cart update/delete** | No `PATCH`/`DELETE` on cart items, no clear-cart endpoint. | **Small** |
| **Order cancellation** | No self-service or admin cancel endpoint. Inventory-restoration-on-cancel logic doesn't exist yet. | **Small–Medium** |
| **Product edit/delete** | No `PATCH`/`DELETE` on products, no add-variant-to-existing-product endpoint. | **Small** |

**Rough sizing key:** Small ≈ a day or less per endpoint once the pattern is established (most of these reuse the exact transactional/validation patterns already proven in the existing codebase). Medium ≈ multiple related endpoints + a new table + real business logic. Large ≈ a multi-step stateful workflow spanning several endpoints and admin UI.

### 3. Known open bug

- **Production 500 on sign-in** (Vercel-hosted) — was being actively debugged, paused mid-diagnosis to prioritize payments work. Root cause not yet confirmed. This needs to be closed before launch regardless of anything else, since it blocks the entire live deployment.

### 4. Frontend — storefront

**Status: 0% built.** Every page — product listing, product detail, cart, checkout, order history, return request UI, account pages — needs to be built from scratch. All backend testing so far has been via Postman; no UI has been written.

This is the single largest remaining body of work in absolute size, even though each individual piece is conceptually straightforward (the backend it talks to already exists for most of the "happy path").

### 5. Frontend — admin dashboard

**Status: 0% built.** Same situation — the APIs exist for products/inventory/orders/dashboard, but there is no actual interface: no login screen, no product form, no order table, no notification bell, no returns-review screen.

### 6. Frontend — mobile

**Status: correctly deferred to v2 per the PRD.** Not a gap against the current plan — this is intentionally out of v1 scope, not behind schedule.

### 7. Testing

**Status: 0% automated.** The Standard of Operations now requires automated tests for all new work, plus rollback/idempotency tests for transaction and webhook logic specifically. None of the existing, already-built routes currently have automated test coverage — they were validated manually via Postman during development. Retrofitting tests onto the existing routes is itself a body of work, separate from writing tests for new endpoints going forward.

### 8. Architecture items not yet applied

- SSR/ISR — moot until the storefront exists to apply it to.
- TanStack Query + Zustand — moot until frontend work starts.
- Guest cart schema change — see §2, not yet applied to the actual database.

---

## Overall Verdict

**The remaining work is large, not minimum**, specifically because:

1. **The entire frontend (storefront + admin dashboard) is unbuilt** — this is likely 60–70% of the total remaining effort by itself, regardless of backend completeness.
2. **Returns, refunds, and deliveries are entirely new subsystems** — not extensions of existing code, but new tables, new multi-step state machines, and new admin UI, none of which exists today.
3. **The payment webhook — arguably the most urgent single gap** — is comparatively small in code size but blocks the core value proposition (an order isn't really "paid" until this exists), so it should not be mistaken for low-priority just because it's not a large body of work.
4. **Automated testing is a new process requirement** applying both retroactively (existing routes) and going forward (everything new) — this adds real time to every subsequent PR that the original backend build didn't have to account for.

## Recommended Priority Order (given the 1-month, 3-person target from the PRD)

Matches the risk mitigation already flagged in the PRD (§9): ship the core loop solid, treat the more complex workflows as things to harden immediately after a soft launch rather than gates that block it.

1. **Payment webhook** (urgent, small-medium, unblocks the entire payment story)
2. **Fix the production sign-in bug** (blocking, unknown size until root-caused)
3. **Storefront UI — core path only** (browse, cart, checkout, order confirmation) — in parallel with #1/#2
4. **Admin dashboard UI — core path only** (products, inventory, orders, dashboard — the parts with working APIs today)
5. **Order cancellation + manual payments** (small-medium, rounds out the payment/order story)
6. **Categories/addresses/cart CRUD gaps** (small, but needed for the storefront UI to actually function — a customer can't add a delivery address without this)
7. **Guest cart** (medium — genuinely improves conversion, but the system functions without it if account-required checkout is an acceptable stopgap for a soft launch)
8. **Returns, refunds, deliveries, notifications** (large, and per the PRD's own risk section, the most defensible thing to launch without and add immediately after — handle manually/off-system in the interim if the 1-month mark arrives before these are ready)
9. **Automated test coverage** — should be applied to everything built from this point forward (per the Standard of Operations), with retroactive coverage of the existing routes as time allows rather than a hard blocker

This ordering gets a genuinely sellable v1 (customer can browse, pay, receive an order; admin can manage stock and fulfill orders) live fastest, while being explicit that returns/refunds — the most complex, least-started subsystem — is the correct thing to hold back rather than the storefront or payment confirmation.